Tutorial 8

Introduction to Distributed-Memory Programming using MPI

CS3210 – 2026/27 Semester 1

Learning Outcomes

1. Identify and solve synchronization problems in communication
2. Learn distributed-memory parallel programming via MPI with C/C++
3. Learn how to compile and run an MPI program
4. Learn how to map MPI processes on different nodes and cores
5. Learn blocking and non-blocking process-to-process communication

This tutorial consists of **two parts**, like tutorial 5. The first part consists of exercise questions, and the second part is a guided practical, in which you will learn how to get started with writing MPI code.

This tutorial has a graded submission. Make sure to read carefully and submit by the deadline. Full marks can be obtained for submissions that have minor inaccuracies.

MPI (Message Passing Interface) is a standardized and portable programming toolkit for programming distributed-memory systems using message passing. It is supported by a few programming languages as an external library (there are many different implementations of the MPI standard).

MPI Standards and Implementations

MPI (Message Passing Interface) is a **message passing library standard** based on the consensus of the MPI Forum which comprises many commercial and research organizations and personnel. The MPI Forum first released the MPI-1 standard in 1992, which was followed by MPI-2 (2000), MPI-2.1 (2008), MPI-2.2 (2009), MPI-3 (2012), MPI-3.1 (2015), and MPI-4 (2021) (see the MPI Forum documents for more details).

Please do not confuse the MPI standard with MPI Implementations. The MPI standards above are implemented differently by different organizations such as MPICH, IBM-MPI, and OpenMPI, among others. In our labs we are using OpenMPI, which is an open-source implementation of the MPI standard. **OpenMPI version 4.1.2** (with full support for the MPI-3 standard) is installed on the lab machines. You can print the version information by running `ompi_info` on the command line (terminal).

> ℹ **Logging in & Getting Started**
>
> Follow the instructions from previous (non-GPU) labs / tutorials to `ssh` into one of our lab machines. Remember that you can still refer to https://guide.cs3210.org. Now, download the code from https://comp.nus.edu.sg/~cs3210/T8_code.zip.

Part 1: Exercise Questions

1. **[MPI]** Consider the following incomplete piece of an MPI program:

```
int rank, p, size=8;
int left, right;
char send_buffer1[8], recv_buffer1[8];
char send_buffer2[8], recv_buffer2[8];
...

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &p);
left = (rank-1+p) %p;
right = (rank+1) %p;
...

MPI_Send(send_buffer1, size, MPI_CHAR, left, ...);
MPI_Recv(recv_buffer1, size, MPI_CHAR, right, ...);

MPI_Send(send_buffer2, size, MPI_CHAR, right, ...);
MPI_Recv(recv_buffer2, size, MPI_CHAR, left, ...);
...
```

   a. In the program, the processes are arranged in a *logical ring* and each process should exchange its name with its neighbor to the left and its neighbor to the right. Assign a unique name to each MPI process and fill out the missing pieces of the program such that each process prints its own name as well as its neighbors' names.

   b. In the given program, `MPI_Send()` and `MPI_Recv()` operations are arranged such that depending on the implementation a deadlock can occur. Describe how a deadlock may occur.

   c. Change the program to get the same behavior but without deadlock while still using only `MPI_Send()` and `MPI_Recv()`.

   d. Change the program such that `MPI_Isend()` and `MPI_Irecv()` are used.

Part 2: Writing MPI Programs with Point-to-Point Communication

⚠️ **Important: Telling MPI Which Network Interface to Use**

MPI needs to know which network interface to send messages on, and our `soctf` machines have multiple. Please do this before running the commands in this tutorial or the commands may not work:

1. Run the line: `export OMPI_MCA_btl_tcp_if_include=172.28.179.137/24` in each terminal you use, or
2. Add that line to your `.bashrc` or `.bash_profile`

We recommend typing the line rather than copy-pasting it from the PDF, as it may not copy correctly.

Compiling and Running MPI Programs Locally

An MPI program consists of multiple processes cooperating via a series of MPI routine calls. Each MPI process has a unique identifier known as its *rank*. Ranks are used by the programmer to specify the source and destination of messages as well as to conditionally control the program execution (e.g., `if (rank == 0) { // do this } else { // do that }`). Processes can communicate with each other using *communicators*. By default, all MPI processes in a program have access to the global communicator `MPI_COMM_WORLD`.

Let's look at the `hello.cpp` program to identify the basic elements of an MPI program:

>..

- Open the code in a terminal (console). You may use any available text editor:

  ```
  > vim hello.cpp
  ```

- **Study the MPI calls in `hello.cpp` (those that start with `MPI_`).**
- To compile this program, use the command `mpic++`:

  ```
  > mpic++ hello.cpp -o hello
  ```

- To run the program with 4 processes, use the command `mpirun`:

  ```
  > mpirun -np 4 ./hello
  ```

`mpirun` is a script that executes MPI programs. We use two of its parameters:

1. The number of processes to be created, `-np <n>`, where n is an integer.
2. The path to the program binary. In this case, `./hello`

✎ **Exercise 1**

Compile and run the `hello.cpp` program locally (i.e., without Slurm) on an soctf lab machine as shown above, with 1, 4 and 32 processes, and observe the output.

- How are the "hello world" messages ordered?
- Is there a limit to the number of MPI processes you can launch on your single node? What if you use the `--oversubscribe` option?

Let's try to quickly run our MPI code on multiple nodes "manually".

> ✎ **Exercise 2**
>
> - Make sure your SSH key is copied to `soctf-pdc-001` and `002` for this exercise. If not, run: `ssh-copy-id soctf-pdc-001` (it'll be present in the other nodes automatically due to NFS)
> - Now run the following command to run `hello` on `soctf-pdc-001` and `002`:
>
>   ```
>   mpirun -np 4 -H soctf-pdc-001:2,soctf-pdc-002:2 ./hello
>   ```
>
>   Notice that we distributed the tasks across the two hosts we specified.

Running MPI Programs across Multiple Nodes using Slurm

However, the previous exercises do not show what makes MPI interesting, which is its ability to run programs **across many nodes, easily!**

As you may have noticed, the previous method of distributing MPI processes among cluster nodes has a number of drawbacks.

- Users have to tediously distribute SSH keys to all nodes participating in the computation.
- Users have to specify *exactly* which nodes they want to use and the exact socket-core allocation, which is tedious and unscalable.
- It is difficult to measure performance accurately due to non-exclusive reservation of nodes.

These are **problems that Slurm can solve**, as long as Slurm and MPI are properly configured together. If you check the output of `ompi_info`, you will see that our OpenMPI implementation is compiled with the `--with-slurm` flag, and if you check `srun --mpi=list`, you will see a number of plugins that help Slurm recognize and run MPI programs. By default, our deployment of Slurm will use the PMIx process manager.

Let's run our MPI program with Slurm!

> >..
>
> - Running on one node with 4 MPI processes.
>
>   ```
>   > srun --nodes 1 --ntasks 4 ./hello
>   ```
> - What if we want to run it on *many* more nodes?
>
>   ```
>   > srun --nodes 4 --ntasks 4 ./hello
>   ```
> - What if we want to specify that each node runs 2 tasks (2 MPI processes)?
>
>   ```
>   > srun --nodes 4 --ntasks-per-node 2 ./hello
>   ```
> - What if we want to run these tasks on a specific node type?
>
>   ```
>   > srun -p i7-7700 --nodes 1 --ntasks 8 ./hello
>   ```

> ✎ **Exercise 3**
>
> Use the last command above to try to find out the maximum number of tasks (MPI processes) that you are allowed to run on a **single node** – try all the different partition types. Does this depend on the number of *physical* or *logical* CPU cores on that node? Try to remove this restriction using `srun`'s `--overcommit` option.

This co-operation between Slurm and MPI allows for a separation of concerns: MPI programs can focus on communication between each other, and Slurm can handle job scheduling and resource allocation.

Mapping MPI Processes to Nodes / Cores

So far, we have let Slurm automatically decide how to map each of our MPI processes to the cluster's nodes and the CPUs on each node. However, we may need more control to get better / more predictable performance.

Distributing MPI Processes to Nodes

There are multiple ways to distribute MPI processes to nodes. The most common methods are *block* (default), *cyclic*, *plane*, and *arbitrary*.

Method	Description
Block	The default method. MPI processes are distributed in a block fashion, i.e., the first n processes are allocated to the first node, the next n to the next node, and so on.
Cyclic	MPI processes are distributed in a cyclic fashion, i.e., the first process is allocated to the first node, the second to the second node, and so on. When the last node is reached, the next process is allocated to the first node.
Plane	MPI processes are distributed in a block-cyclic fashion: processes are split into blocks of a maximum size, and then distributed cyclically across nodes.
Arbitrary	MPI processes are distributed in an arbitrary fashion, i.e., the user specifies the distribution of the processes to the nodes.

Table 1: Common MPI Process Distribution Methods

Let's try these methods.

>..

- Block distribution:

```
> srun -N 3 -n 8 --distribution=block --exclusive ./hello
```

- Cyclic distribution:

```
> srun -N 3 -n 8 --distribution=cyclic --exclusive ./hello
```

- Plane distribution:

```
> srun -N 3 -n 8 --distribution=plane=2 --exclusive ./hello
```

- Arbitrary distribution:

```
> SLURM_HOSTFILE=slurm_hostfile.txt srun -n 4 ./hello
```

✎ **Exercise 4**

Observe the spread of tasks in `cyclic` distribution and compare it to `block` and `plane`. Try different `plane` sizes. You may want to pipe it to `sort` to see the results more clearly.

Now, take a look at the contents of the provided file `slurm_hostfile.txt` and run the arbitrary distribution command. The presence of the `SLURM_HOSTFILE` environment variable automatically sets `--distribution=arbitrary` so you don't have to set it manually. How do the contents of `SLURM_HOSTFILE` relate to how the tasks are distributed?

Try to increase and decrease the number of tasks and observe the results. Also, try changing the contents of `slurm_hostfile.txt` and confirm that it works as you expect.

Distributing MPI Processes to CPUs Within a Node

Now that we have full control over distributing tasks to nodes, we can also control how we would like the tasks *within a node* to be allocated to the *CPUs on that node*. We discussed this briefly in Exercise 2 with the `--bind-to` option and Exercise 4 with the `rankfile`, but since we can no longer run `mpirun` directly, we will do this through Slurm. This is much more powerful and less tedious.

We first need a consistent way to say exactly which cores / threads within a processor we want to force a task to run on. This set of CPUs for a task is known as a `cpuset`, which is just a hexadecimal number such as `0x11`. For consistency, we will focus on the `i7-7700` nodes.

>..

- Let's observe the list of cores and their `cpuset` within an i7-7700 machine.

  ```
  > srun --partition i7-7700 lstopo-no-graphics --taskset
  ```

- Focus only on the lines that either start with `Core` or `PU`. `Core` refers to one physical core, whereas the two `PU`s within the `Core` each refer to one hardware thread because this processor supports hyperthreading.
- Looking at `Core L#0`, we see that it has a cpuset of `0x11`, which is just a combination (bitwise *or* operation) of cpusets of its hardware threads `PU L#0` (`cpuset=0x1`) and `PU L#1` (`cpuset=0x10`).
- Therefore, if we set the cpuset for a task to `0x1`, it will *only run on hardware thread (PU) L#0*. Likewise, if we set it to `0x11`, it is free to run on either of the hardware threads of `Core L#0`. The most permissive cpuset for this processor is `0xff` (all hardware threads allowed) – you can see this from the top level `NUMANode` line.

✎ **Exercise 5**

Now that we understand cpusets, we can look at the output of the *cpu bindings* that Slurm can do for our tasks. Try these commands:

```
> srun -n 4 -p i7-7700 --cpu-bind=verbose,threads ./hello
```

```
> srun -n 4 -p i7-7700 --cpu-bind=verbose,cores ./hello
```

Based on the cpusets (masks) that are reported for each task, what can you conclude about how each of these Slurm `cpu-bind` options binds each MPI process to the node's CPUs?

Explore the `cpu-bind` section in the srun documentation page to see the description of each option and all the other available options.

Note: this general concept of binding a task to particular cores is called *CPU affinity*. It can be set by MPI without Slurm, and can even be set for any normal executable (see the `taskset` command).

Process-to-process Communication

While running completely separate tasks on many machines might be useful, the true power of MPI is in its ability to handle complex communication between each of its processes.

MPI provides two types of process-to-process communication: blocking and non-blocking. Each of these communication types is accomplished with a send and a receive function.

Blocking Communication

A blocking send stops the caller until the message has been copied over to the underlying communication buffer (i.e. network buffer). Similarly, blocking receive blocks the calling process until the message has been received in the MPI process.

The function signatures for a blocking send and receive are respectively:

```
</>  int MPI_Send(const void *buf, int count, MPI_Datatype datatype,
                 int dest, int tag, MPI_Comm comm)
```

```
</>  int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
                 int source, int tag, MPI_Comm comm, MPI_Status *status)
```

Table 2 lists the arguments to be used with blocking communication functions.

buf	Pointer to the memory buffer that holds the contents of the message to be sent or received
count	The number of items that will be sent.

datatype	Specifies the primitive data type of the individual item sent in the message, and can be one of the following:
	`MPI_CHAR`
	`MPI_SHORT`
	`MPI_INT`
	`MPI_LONG`
	`MPI_UNSIGNED_CHAR`
	`MPI_UNSIGNED_SHORT`
	`MPI_UNSIGNED_LONG`
	`MPI_UNSIGNED`
	`MPI_FLOAT`
	`MPI_DOUBLE`
	`MPI_LONG_DOUBLE`
	`MPI_BYTE`
	`MPI_PACKED`

dest/source	Specifies the rank of the destination / source process in that MPI communicator
tag	An integer that allows the receiving process to distinguish a message from a sequence of messages originating from the same sender
comm	The MPI communicator
status	Pointer to an `MPI_Status` structure that allows us to check if the receive has been successful.

Table 2: Arguments for Blocking Communication Routines

> ✎ **Exercise 6**
>
> Open the program `block_comm.cpp` and observe the series of `MPI_Send` and `MPI_Recv` calls.
>
> Compile `block_comm.cpp` (refer to Exercise 1 if necessary) and use `srun` to run it with *two processes* across *two different nodes*.

> ✎ **Exercise 7**
>
> Modify the file `block_comm.cpp` (new name `block_comm_1.cpp`) such that process 1 sends back to process 0 ten floating-point values in one message. Compile the program and run it with two processes across two different nodes.

> ✎ **Exercise 8**
>
> What happens if we flip the order of `MPI_Send` and `MPI_Recv` in the master process? Consider the implications of this.

Non-blocking Communication

Non-blocking communication, in contrast to blocking communication, does not block either the sender or the receiver.

The function signatures for a non-blocking send and receive are respectively:

```
</>  int MPI_Isend(const void *buf, int count, MPI_Datatype datatype,
                  int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

```
</>  int MPI_Irecv(void *buf, int count, MPI_Datatype datatype,
                  int source, int tag, MPI_Comm comm, MPI_Request *request)
```

The only new parameter in the call is:

request	Pointer of type `MPI_Request` which provides a handle to this operation. Using this handle, the programmer can inquire later whether the communication has completed.

Both functions return immediately, and the communication will be performed asynchronously w.r.t. the rest of the computation. Using these functions, the MPI program can overlap communication with computation. It is unsafe to modify the application buffer (your variable) until you know for a fact the requested non-blocking operation was actually performed by the library.

To check whether the functions have finished communicating, we can use:

```
</>  int MPI_Test(MPI_Request *request, int *flag, MPI_Status *status)
```

`MPI_Test` takes a handle of an `MPI_Isend` or `MPI_Irecv` and stores a true/false in the flag variable which indicates whether the operation has been completed.

```
</>  int MPI_Wait(MPI_Request *request, MPI_Status *status)
```

`MPI_Wait` blocks the current process until the request has finished.

MPI provides quite a few variants of these basic two functions. You are encouraged to read the MPI manual for these functions:

```
MPI_Test, MPI_Testall, MPI_Testany, MPI_Testsome
MPI_Wait, MPI_Waitall, MPI_Waitany, MPI_Waitsome
```

MPI does **NOT** guarantee fairness, thus the programmer should make sure that starvation does not occur.

Exercise 9

Compile the program `nblock_comm.cpp` and run it with *3 processes* across *3 nodes*. Modify the source (new name `nblock_comm_1.cpp`) code to swap the order of the `MPI_Isend` and `MPI_Irecv`. Compile and run it again. What do you observe?

More information and further reading:

- Communication in a ring: Correctness and Performance. https://www.epcc.ed.ac.uk/whats-happening/articles/what-mpi-nonblocking-correctness-and-performance

For Submission (Exercises 10-12)

Exercise 10

View and compile the MPI matrix multiplication program, `mm-mpi.cpp`.

This program implements distributed matrix multiplication. The matrix size must be a power of 2, and the total number of processes must be of the form ($2^n + 1$ for some n).

Your goal in this exercise is to run `mm-mpi` with a matrix size of **2048** using Slurm, across a few configurations, and **explain the differences in performance you see**.

Use these configurations only (but you may add options for debug reasons):

- Run on **one** `i7-7700` node with 5 total processes and include the option `--cpu-bind=thread`.
- Run on **one** `xs-4114` node with 5 total processes and include the option `--cpu-bind=thread`.
- Run on **one** `i7-7700` **and one** `xs-4114` node with 5 total processes with a cyclic distribution and `--cpu-bind=thread`. You may need the Slurm option `--constraint="[i7-7700*1&xs-4114*1]"`.

Refer to the communication and computation time reported by the worker processes, and the total time reported by the master process.

✎ **Exercise 11**

There's an unexpected and significant performance difference between these two commands:

- `srun -n5 -N1 -p i7-7700 ./mm-mpi 256`
- `srun -n9 -N1 --overcommit -p i7-7700 ./mm-mpi 256`

Run and explain the performance gap between these two configurations. You can assume that they both compute the correct result. You may add code such as `printf` statements for debugging.

✎ **Exercise 12**

Suggest how you could change `mm-mpi.cpp` to fix the slow performance of the overcommitting configuration, while maintaining correctness. You should not limit the number of MPI processes that participate in the program, or use other libraries like OpenMP/CUDA. You do not have to submit any code.

! **Tutorial submission:**

Submit the report in **PDF** form with file name format A0123456X.pdf (**Do not zip your file!**) via Canvas before **Monday 26 October, 2pm** (note: not 11.59 pm in this course!).

The document must contain:

- A maximum of 1 page.
- Your explanation/data/visualization for exercise 10
- Your explanation/data/visualization for exercise 11
- A brief description of your idea for exercise 12 (you don't have to submit code), and why it works.
- Any information that you need to include in the appendix